

http-api

HTTP API Contract — Delta for Chapter Hierarchy

Feature: specs/003-chapter-hierarchy | **Date:** 2026-09-05

This is a **delta**. specs/001-workshop-curriculum-platform/contracts/http-api.md remains authoritative for everything it already covers — the response envelope, the three-state contract, the header set, the error taxonomy, the malformed-request rules — and this document restates none of it, so the two cannot drift.

This feature adds NO endpoint. §3 changes two that exist. There is no §“Endpoints added”, and its absence is deliberate rather than an omission: see §5.

1. Binding rules inherited, not restated

Every response below inherits, without exception:

1. The **three states** — ok, no_match, unavailable — and the rule that a degraded backend is never rendered as an empty result set.
2. The **four resolution outcomes** — found, redacted, not present, could not determine — mapped to their existing status codes.
3. **Malformed requests are a fourth thing**, not a fourth state, and use the existing 4xx error shape rather than borrowing the three-state vocabulary.
4. The **generation header** and Cache-Control: no-store on the three-state endpoints.
5. X-Workshop-Search-Status mirrors body.status verbatim and is never the authority.

2. The two amendments to the 001 contract

Both are **prerequisites**, not consequences. Neither may be enacted by implementing it, and every field marked (*provisional*) below is provisional on the first.

- **A1 — Chapter.ordinal is widened to ordinal_path.** specs/001-.../data-model.md:45 types ordinal as int. 02.01 has no int ordinal, and ordinalOf (internal/api/chapters.go:736) faithfully returns 2 for both 02 and 02.01. The field becomes an ordered list of integers. **ordinal is removed, not deprecated:** leaving it beside ordinal_path guarantees a client reads the one that collides.
- **A2 — http-api.md:60 is amended to name the dotted id as the path key.** It currently reads that the zero-padded ordinal is “*not accepted as a path key — one key, one meaning*”, while route-manifest.tsv:72 declares the live route GET /api/chapters/01 — which is exactly that string. The amendment makes the **id** the single key, so “*one key, one meaning*” becomes true rather than merely written. {chapter} continues to match one path segment; a chapter id contains no / because SafeSlug (pkg/curriculum/curriculum.go:209) rejects it.

3. The route-manifest obligation

The server-unity verifier reads the route manifest and this contract and fails when a contracted endpoint has no manifest row. Its exit is three-valued; an undetermined result is **not** a pass.

- **R1 — both endpoints in §4 already have rows. Their contract citation must name both documents** as a compound token — 3.2+003.4.1 and 3.3+003.4.2 — because a row naming only the 001 section still looks complete to a reviewer while the behaviour underneath it has changed. This is the same staleness R1b guards for feature 002, and it is a *stale* row rather than an *absent* one, which no existence check can see.
- **R2 — route-manifest.tsv:64 reads #subst {chapter} 01. One substitution value cannot exercise a hierarchy.** The manifest must substitute at least a parent and a child, and the rows written with a literal /api/chapters/01/ — :72–:76, :85, and the four NOT_BUILT rows at :157–:160 — must be re-expressed against {chapter} so the substitution reaches them.
- **R3 — no row is added.** §5.

4. Endpoints changed

4.1 GET /api/chapters — the list, with derived ancestry

Today it returns {"chapters": [{ "id": ..., "material_count": ...}]} and nothing else (cmd/workshop-server/main.go:2078–2091). The array is already sorted byte-lexicographically on id (:2126).

Changed: every row gains a hierarchy object; the endpoint gains three query parameters and a filters echo. **The chapters value stays a FLAT ARRAY** — core/api.ts:88 reads it flat, and nesting it is a breaking change to a live consumer.

```
// 200 – GET /api/chapters
{
  "status": "ok",
  "filters": {                                // ECHOED ON EVERY STATUS. See
C4.1.4.
    "under": null,
    "depth": null,
    "include_self": false,
    "under_resolved": null                  // null when `under` was not
given
  },
  "max_depth_present": 2,                    // MEASURED, never a limit.
H6.
  "unclassified": [],                       // H1. See C4.1.6.
  "chapters": [
    {
      "id": "01",
      "material_count": 41,
      "hierarchy": {
        "parent_id": null,
        "depth": 1,
        "ancestor_ids": [],
        "child_ids": [],
        "ordinal_path": [1],                // (provisional on A1)
        "orphaned": false
      }
    },
    {
      "id": "02",
      "material_count": 20,
      "hierarchy": {
        "parent_id": null,
        "depth": 1,
        "ancestor_ids": [],
        "child_ids": ["02.01"],
        "ordinal_path": [2],
```

```

        "orphaned": false
    }
},
{
    "id": "02.01",
    "material_count": 5,
    "hierarchy": {
        "parent_id": "02",
        "depth": 2,
        "ancestor_ids": ["02"],
        "child_ids": [],
        "ordinal_path": [2, 1],
        "orphaned": false
    }
}
]
}

```

C4.1.1 — the array is flat and stays flat. No element carries a nested chapter array. Ancestry travels per row. A client that wants a tree builds one from `parent_id`; a client that wants a list keeps working unchanged.

C4.1.2 — order is byte-lexicographic on id, and it is the same order the filesystem and a shell glob produce. Not a coincidence: it is asserted (H4). 02.01 falls immediately after 02 because . (0x2E) sorts below 0–9 (0x30–0x39).

C4.1.3 — every hierarchy field is derived and none is stored (H2). A client may recompute all of them from the id list alone and must get the same answer; a gate does exactly that.

C4.1.4 — filters is echoed on EVERY status, including unavailable. This is the load-bearing rule of this section. Without the echo, four situations are indistinguishable to a client: the branch is empty; the parameter was misspelled; the parameter was ignored by an older build; the backend could not read the tree. The first is a fact about the corpus and the other three are faults, and a client that cannot tell them apart will render the fault as the fact.

// 503 – the chapter tree could not be read. FILTERS ARE STILL ECHOED.

```

{
    "status": "unavailable",
    "generation": null,
    "reason": {
        "code": "curriculum_unreadable",
        "leg": "curriculum",
        "message": "The chapter tree could not be read, so no

```

```

hierarchy can be derived.",
  "retry_after_s": null
},
"filters": { "under": "02", "depth": null, "include_self":
false, "under_resolved": null }
}

```

Note `under_resolved` is **null**, not `false`, here: `false` asserts that the branch was looked for and not found, and nothing was looked for. Collapsing the two is the same conflation the platform's fourth resolution outcome exists to prevent.

C4.1.5 — the three query parameters.

Parameter	Form	Behaviour
under	a chapter id	returns the descendants of that id, at every depth. Filters on derived ancestry , so an orphan whose parent is absent is still returned under that parent's id (C4.2.3)
depth	a positive integer	returns only chapters at exactly that depth. Combines with <code>under</code> by intersection
include_self	boolean	when <code>under</code> is given, includes the named chapter itself. Default <code>false</code>

An unparseable `depth` or `include_self` is a **malformed request** — 4xx, the existing error shape, no three-state vocabulary. It is not defaulted. The precedent is `boolParam` (`internal/api/chapters.go:819–821`), which already rejects rather than defaults, and its reasoning applies unchanged: a silently defaulted filter returns a correct-looking answer to a question nobody asked.

C4.1.6 — an unknown under is 200, never 404.

```

// 200 – GET /api/chapters?under=99
{
  "status": "ok",
  "filters": { "under": "99", "depth": null, "include_self":

```

```

false, "under_resolved": false },
  "max_depth_present": 2,
  "unclassified": [],
  "chapters": []
}

```

The route exists and the request was well-formed. A 404 here says “*no such endpoint*” about a request whose true answer is “*no such branch*”. `under_resolved: false` is what carries the second meaning, and it is why the empty array is not ambiguous.

C4.1.7 — unclassified reports what the grammar refused (H1). A directory under `chapters/` whose name does not match `^[0-9]{2,}(\.[0-9]{2,})*$` appears here with a reason. It is **never** silently skipped and **never** silently listed as a chapter.

```

"unclassified": [
  { "name": "02.2",      "reason": "component not zero-padded to at
least two digits" },
  { "name": "drafts",    "reason": "not a numeric path" }
]

```

The reason is a **classification**, not a filename echo beyond the directory’s own name, which the existing list endpoint already returns as `id.listChapters` (`main.go:2094`) deliberately counts materials rather than naming them; that discipline is unchanged, and nothing in this feature puts a material filename on the wire.

4.2 GET /api/chapters/{chapter} — one chapter, with parent and child hrefs

Changed: `ordinal` → `ordinal_path` (A1); title no longer collides; `derivation.ordinal` rewritten; a hierarchy object added. Everything else — `summary: null`, `materials`, `sections`, `status`, and the existing `derivation` entries for them — is unchanged.

```

// 200 – GET /api/chapters/02.01
{
  "slug": "02.01",
  "ordinal_path": [2, 1],           // (provisional on A1) –
REPLACES `ordinal`, never beside it
  "title": "Chapter 2.1",
  "summary": null,
  "hierarchy": {
    "parent_id": "02",
    "parent_href": "/api/chapters/02",
    "depth": 2,
    "ancestor_ids": ["02"],

```

```

    "child_ids": [],
    "child_hrefs": [],
    "orphaned": false
  },
  "materials": [ /* unchanged */ ],
  "status": "draft",
  "sections": null,
  "derivation": {
    "ordinal_path": "each dot-separated component of the chapter
id, parsed as an integer, in order",
    "title": "positional – no authored chapter title exists in
this curriculum, so the title is the chapter's own ordinal path
and nothing more",
    "summary": "not authored – null rather than a placeholder, and
rather than a generated stand-in: no summary was written and
nothing local can write one",
    "hierarchy": "derived from the chapter id by string operation;
no parent, depth or child list is stored anywhere",
    "materials": { /* unchanged */ },
    "status": "draft – no transcript has been produced for this
chapter",
    "sections": { /* unchanged */ }
  }
}

```

C4.2.1 — derivation.ordinal must be rewritten, not deleted. It currently reads “*parsed from the leading digits of the chapter slug*” (chapters.go:141), which is an **accurate description of the defect** — the endpoint is not lying, it is telling you exactly what it did. That honesty mechanism is the reason the defect is legible at all, and a derivation string left describing a replaced derivation would convert a working instrument into a stale one.

C4.2.2 — title must not collide. chapterTitle(:755) renders 02 and 02.01 both as “*Chapter 2*”. Two distinct chapters with one title is a wrong answer a reader will read as a cosmetic quirk.

C4.2.3 — an orphan is SERVED (H3).

```

// 200 – GET /api/chapters/02.01, with chapters/02 absent
{
  "slug": "02.01",
  "ordinal_path": [2, 1],
  "title": "Chapter 2.1",
  "hierarchy": {
    "parent_id": "02", // UNCHANGED by the parent's
absence – it is derived
    "parent_href": null, // there is nothing to link to
    "depth": 2,
    "ancestor_ids": ["02"],

```

```

    "child_ids": [],
    "child_hrefs": [],
    "orphaned": true,
    "missing_ancestor_ids": ["02"]
  }
}

```

Hiding it loses content that exists. Promoting it to a root silently rewrites the hierarchy. Serving it with `orphaned: true` and the missing id named is the only form that states what is true.

C4.2.4 — the four outcomes are unchanged. A chapter that does not exist is 404 with the existing error shape; a tree that cannot be read is 503 with `status: "unavailable"`; neither is collapsed into the other. Orphanhood is **not** a fourth outcome — the chapter was found.

5. Why no route is added

The obvious design is `GET /api/chapters/{chapter}/children`. It is refused, and the reason is a cost that is easy to miss: a new route is not one handler. It is a route-manifest row, a section in this contract, a server-unity verifier row, a negative assertion for the not-built case, and a permanent line in every future audit of the API surface — and the manifest already carries **20** `NOT_BUILT` rows, which is the measured evidence that declared-but-unbuilt routes accumulate.

Every fact such a route would serve is derivable from data `GET /api/chapters` already returns. Adding one would create a second way to ask the same question, and the two would be free to disagree the first time a filter was added to one of them.

6. Gates and their paired mutations

Every gate is three-valued — 0 clean, 1 a real finding, 2 could not determine — and every one takes its chapter ids **from the live tree**, following `verify-absence-honesty.sh:127–134`, which already reads a chapter from `GET /api/chapters` rather than hardcoding one.

Gate	Asserts	Paired mutation that must turn it red
G-CH-1 grammar	the validator accepts 01 02 02.01 02.01.01	relax the grammar to [0-9]+

Gate	Asserts	Paired mutation that must turn it red
	100 and rejects 2 02.2 02. .01 ""	
G-CH-2 three orders	API order, shell-glob order and filesystem byte order are identical over a fixture tree	sort by ordinal_path numerically — must go red on 02.09 / 02.10
G-CH-3 derived-only	no persisted parent_id, depth or child list in the registry, the derived database or any serialised artifact	persist one
G-CH-4 no collision	ordinal representations are pairwise distinct over every present id plus a synthetic set	restore leading-digit parsing — must go red on 02 / 02.01
G-CH-5 scope equality	no chapter-scope comparison uses HasSuffix, HasPrefix or Contains	introduce one
G-CH-6 two-scope redaction	the review for 01 excludes 02.01's artifacts, on a two-scope fixture	restore strings.HasSuffix
G-CH-6b fixture blindness	the same mutation against a one-scope fixture leaves the test green	— this gate asserts a <i>negative</i> about a test, and has no mutation of its own; it is the mutation
G-CH-7 filters echo	filters is present on ok, no_match and unavailable	omit it on the unavailable path
G-CH-8 unknown branch	an unknown under returns 200 with under_resolved: false	return 404

Gate	Asserts	Paired mutation that must turn it red
G-CH-9 orphan served	an orphan is listed, flagged, and names its missing ancestor	filter orphans out of the list
G-CH-10 no special case	no branch anywhere conditions on a chapter's depth	add if depth > 1 to any handler
G-CH-11 flat array	chapters is a flat array in every response	nest children under their parent
G-CH-12 both directions	both widened patterns match chapter-01 and chapter-02.01	(a) revert to chapter-\d+; (b) narrow to chapter-\d+\.\d+. Both required
G-CH-13 ingest refusal	ingestion with a chapter and no transcript export refuses , writing zero records	restore the full_ch01 default
G-CH-14 no hardcoded chapter in gates	every gate obtains its chapter from the live tree or from an argument	hardcode 01 in one

G-CH-6b is the unusual one and is stated rather than folded in. It asserts that a cheaper test *fails to detect* the defect. Without it, a future maintainer replacing the two-scope fixture with a one-scope one sees every assertion still pass and concludes the fixture shape was incidental. It was not: 02.01 has the suffix 01, so on a one-scope registry HasSuffix and == agree on every constructible input, and the narrowing fix is a **no-op** there.

7. Traceability

Contract clause	Requirement	Criterion
A1, C4.2.1, C4.2.2	FR-022, FR-023, FR-024	SC-013
A2, R2	FR-002, FR-033	SC-015, SC-025

Contract clause	Requirement	Criterion
C4.1.1, C4.1.2	FR-009, FR-010, FR-011, FR-027	SC-003, SC-004, SC-016
C4.1.3	FR-006, FR-007	SC-011, SC-012
C4.1.4	FR-028	SC-017
C4.1.5	FR-028	SC-017
C4.1.6	FR-029	SC-018
C4.1.7	FR-004, FR-005	SC-001, SC-002
C4.2.3	FR-031	SC-019
§5	FR-026	SC-015
§6 G-CH-5, G-CH-6, G-CH-6b	FR-012, FR-013	SC-008, SC-009, SC-010
§6 G-CH-10	FR-032	SC-020
§6 G-CH-12	FR-014	SC-006
§6 G-CH-13	FR-016, FR-017	SC-007

8. Unverified register

Recorded rather than answered, because each would be a guess.

- **U1 — the material_count values in §4.1’s example are measured for the LIST endpoint only.** Counting non-directory, non-dotfile entries — which is what `listChapters` (`main.go:2111–:2123`) does — gives **41, 20** and **5** on 2026-09-05. The single-chapter endpoint’s materials array is **not** the same set: `listMaterials` classifies by extension and excludes the recording, its archive members and its checksum manifest (`chapters.go:156–163`). **The two numbers are not expected to agree**, and neither is a count of “how much material a chapter has”.
- **U2 — whether GET /api/chapters carries a status field today.** It returns a bare `{"chapters": ...}` at 200 and a `{"error": ...}` at 500 (`main.go:2085–2091`); it does not use `writeUnavailable`. §4.1’s `status: "ok"` and the 503 example therefore describe the **target** shape under FR-028, and bringing this endpoint onto the three-state envelope is part of the work, not a description of it.
- **U3 — the maximum depth this corpus will reach.** `max_depth_present` is measured at **2** today. No limit is asserted (H6), and no figure here should be read as one.